

Lux: Current Architecture Specification

Visual Output Surface for AI Agents — a System in Transition

July 2026 — v0.19.0

Contents

1	Overview	2
1.1	Scope	2
1.2	Companion Documents	2
1.3	Design Principles	2
2	System Architecture	3
2.1	Component Map	3
2.2	Data Flow	4
2.2.1	Write Path (Agent → Display)	4
2.2.2	Event Path (User → Hub, two-tier D21)	5
2.2.3	Update Path (Incremental Patching)	5
3	The Hub/Display Slice (Migrated Core)	5
3.1	Two Authorities, One Winner	5
3.2	HubDisplay: the Authoritative Store	6
3.3	The Element ABC	6
3.4	Two-Tier Handler Dispatch (D21)	6
3.5	Agent Subscribe / Publish	7
3.6	Migrated vs. Still-Legacy	7
4	Wire Protocol	7
4.1	Framing	7
4.2	Message Types	8
4.2.1	Client → Display	8
4.2.2	Display → Client	8
4.3	Element Kinds	9
4.4	Draw Commands	10
5	Display Server Internals	10
5.1	Frame Cycle	10
5.2	Multi-Scene State	11
5.2.1	Scene Lifecycle	11
5.2.2	Widget State Swap Pattern	11
5.3	Table Filtering	12
6	Frame Architecture	12
6.1	Frame Class	12
6.2	Frame Lifecycle	12
6.3	World Menu	13
6.4	Dock Bar	13
7	IPC and Process Management	13
7.1	Socket Discovery	13
7.2	PID Lifecycle	13
7.3	Auto-Spawn	14
7.4	Tool Reconnect Path	14

8	Performance	14
8.1	Frame Budget	14
8.2	Critical Path Analysis	14
8.3	Idle Screen	14
8.4	Memory	15
8.5	Known Limitations	15
9	Security	15
9.1	Threat Model	15
9.2	Render Function Security	15
9.3	Protocol Robustness	16
10	Platform Integration	16
10.1	macOS	16
10.2	Linux	16
11	MCP Tool Surface	17
12	Formal Model	18
13	Test Architecture	19

1 Overview

Lux is a visual output surface for Claude Code agents and local applications. Agents and apps push JSON element trees into a native ImGui window over IPC. The system has four projection surfaces from a single codebase: Python library, CLI, MCP server, and Claude Code plugin.

This document describes the architecture *as the code stands today*. Today's code is mid-migration between two models, and this specification keeps that distinction explicit throughout:

- The **legacy single-process display-server path** (`src/punt_lux/display/`, `src/punt_lux/scene/`, `src/punt_lux/socket\_server.py`) still carries the bulk of the rendering and scene mass. In this path the display process owns the render loop, scene state, widget state, interaction queue, and query surface; agents are clients; all communication is framed JSON over a Unix domain socket; and the loop is single-threaded, polling all I/O non-blocking each frame via `select()`.
- The **new Hub/Display slice** (`src/punt_lux/domain/`) is the working proof of the target model: a Hub-authoritative `HubDisplay` store, an Element ABC on four migrated kinds, a two-tier remote-dispatch event path, and a Hub-scoped publish/subscribe channel. It is narrow but real, and it is where new work converges. See Section §3.

1.1 Scope

This document is the current-architecture record for Lux — the org-standard answer to “what does the code approximately do today?” It is not the rewrite target. The canonical design intent lives in `docs/architecture/target/target.md`; where this document and that target disagree, the target defines the *direction* and this document reports the *present*.

The system is honestly a work in transition. Rather than pretend the legacy path is gone or that the new slice is complete, the sections below label each subsystem as **migrated**, **transitional**, or **legacy** so a reader can tell settled ground from scaffolding.

1.2 Companion Documents

The sibling files in `docs/architecture/` provide narrower supporting views:

- `target/target.md` — the canonical rewrite target. Start there for design intent; return here for current behavior.
- `display-server.tex` — Z specification of the legacy display-server state machine (client lifecycle, scene replacement, patching, event flush). It models the single-process path, not the Hub/Display runtime, and its refinement tests still hold the current display code to that model.
- `workspace-model.tex` — Z specification of frames, client identity, scene-to-frame routing, and World menu ownership.
- `target/introspection-api.md` — the target verification and control surface realized by the unified `QueryRequest/QueryResponse` envelope and the `inspect_scene()`, `list_scenes()`, and related tools. See Section §11.

1.3 Design Principles

- P1. Composable element tree.** Every element kind is small, nestable, and data-driven. If an agent can describe it as JSON, Lux renders it.
- P2. Protocol is the API surface.** The wire protocol is the only interface between agents and the display. No shared memory, no function calls, no Python imports required.
- P3. Single-threaded simplicity.** The display server uses a polling event loop with zero threads. All state is owned by the main thread. No synchronization primitives.
- P4. Incremental by default.** Scenes can be patched by element ID without replacing the entire tree. Table filters run at frame rate with zero MCP round-trips.
- P5. Multi-scene coexistence.** Each `show()` call opens a new tab. Scenes persist until explicitly dismissed or cleared. Same `scene_id` replaces content in-place.

2 System Architecture

2.1 Component Map

The Python package spans four concerns: the protocol layer (the wire contract), the legacy display/runtime layer with its scene-state management and MCP/client adapters, and the newer `src/punt_lux/domain/` Hub/Display slice. The first three are the mass of the shipped data-display product; `src/punt_lux/domain/` is the migrated core described in Section §3.

Module	File or package	Responsibility
Protocol	<code>src/punt_lux/protocol/</code>	Wire framing, the <code>FrameReader</code> streaming buffer, and the <code>message_to_dict</code> / <code>element_to_dict</code> dispatch helpers.
Elements	<code>src/punt_lux/protocol/elements/</code>	25 wire element kinds plus the <code>Patch</code> payload used by <code>UpdateMessage</code> . Four kinds — <code>text</code> , <code>button</code> , <code>checkbox</code> , <code>dialog</code> — subclass the <code>src/punt_lux/domain/element_abc.py</code> <code>Element ABC</code> (data + behavior); the rest are frozen dataclasses. <code>JsonElementFactory</code> decodes the io-model kinds, <code>ElementCodec</code> the remainder.
Messages	<code>src/punt_lux/protocol/messages/</code>	22 message dataclasses across six families — <code>scene</code> , <code>lifecycle</code> , <code>menu</code> , <code>introspect</code> , <code>observer</code> , and <code>remote_invocation</code> — with <code>MessageRegistry</code> as the wire dispatch table. User interactions cross the wire as <code>RemoteEventHandlerInvocation</code> (<code>remote_invocation</code>), not as a broadcast interaction message.
Display	<code>src/punt_lux/display/</code>	ImGui render loop, scene rendering, frame lifecycle, menu management, screenshot capture, and event flush. The core modules are <code>server.py</code> , <code>element_renderer.py</code> , <code>table_renderer.py</code> , <code>menu_manager.py</code> , <code>idle_screen.py</code> , and <code>texture_cache.py</code> .
Scene	<code>src/punt_lux/scene/</code>	Scene graph state. <code>manager.py</code> owns <code>SceneManager</code> , <code>frame.py</code> defines workspace frames, and <code>widget_state.py</code> holds per-scene ImGui control state.
Domain (Hub)	<code>src/punt_lux/domain/</code>	The migrated Hub/Display slice. <code>element_abc.py</code> is the <code>Element ABC</code> (Composite template method, handler registry, property-observer surface). <code>hub/hub.display.py</code> is <code>HubDisplay</code> , the Hub-authoritative element/owner/child store; <code>hub/hub.py</code> is the connection-scoped Agent Subscribe pub-sub Hub. <code>display.py</code> routes <code>Updates</code> to <code>Events</code> and validates remote interactions in <code>interact()</code> . <code>handlers/remote_dispatch.py</code> wraps handlers for the two-tier path. See §3.
Tools (MCP)	<code>src/punt_lux/tools/</code>	FastMCP tool surface. <code>server.py</code> owns the <code>FastMCP</code> instance and session lifecycle. <code>tools.py</code> defines display-facing helpers; <code>subscribe_tools.py</code> defines the separate session pub-sub helpers.
Socket server	<code>src/punt_lux/socket_server.py</code>	<code>SocketServer</code> — Unix-domain socket lifecycle, non-blocking multiplexing of client connections, per-client <code>FrameReader</code> buffers, and <code>send/recv</code> to clients.
Query dispatcher	<code>src/punt_lux/query_dispatcher.py</code>	<code>QueryDispatcher</code> — routes <code>QueryRequest</code> by <code>method</code> string, and owns the ring buffers for recent interaction events (max 200) and recent error log entries (max 100).

Module	File or package	Responsibility
Client	src/punt_lux/client.py	Backward-compat shim that re-exports <code>DisplayClient</code> under the old name <code>LuxClient</code> .
Display client	src/punt_lux/display_client.py	<code>DisplayClient</code> — typed <code>show</code> , <code>update</code> , <code>query</code> , <code>clear</code> , and <code>ping</code> methods, plus <code>on_event()</code> callback registration for <code>RemoteEventHandlerInvocation</code> frames and <code>poll_event()</code> for queued <code>ObserverMessages</code> .
Gateway	src/punt_lux/luxd.py	WebSocket gateway (<code>luxd</code>) and mcp-proxy entry point — the Hub process in the target topology. In the legacy single-process path it is transport and session bootstrap plumbing, not the source of truth for scene state or rendering.
Apps/Beads	src/punt_lux/apps/beads.py	Beads issue browser; loads <code>.beads/issues.jsonl</code> and renders a filterable table scene.
Show	src/punt_lux/show.py	<code>lux show</code> subcommand for pre-built scenes such as the beads board.
Paths	src/punt_lux/paths.py	<code>DisplayPaths</code> — socket discovery, PID lifecycle, and auto-spawn via <code>ensure()</code> . Module-level helpers also resolve the optional gateway’s PID/port paths.
Runtime	src/punt_lux/runtime.py	<code>CodeExecutor</code> and <code>RenderContext</code> — compile and run agent-submitted render functions per-frame, gated by consent.
Config	src/punt_lux/config.py	<code>ConfigManager</code> reads and writes YAML frontmatter in <code>.punt-labs/lux.md</code> ; the live field today is the per-repo display-mode toggle (<code>display = y n</code>).
Hooks	src/punt_lux/hooks.py	Claude Code lifecycle hook handlers (session start, post-bash).
Service	src/punt_lux/service.py	<code>ServiceManager</code> — <code>launchd</code> / <code>systemd</code> lifecycle for <code>luxd</code> .
Remote	src/punt_lux/remote.py	<code>ProxyConfigFile</code> — mcp-proxy WebSocket transport setup.
CLI	src/punt_lux/__main__.py	Typer app with <code>display</code> , <code>service</code> , <code>status</code> , <code>installation</code> , and gateway management subcommands.

2.2 Data Flow

Three primary data paths connect agents to the display:

2.2.1 Write Path (Agent → Display)

1. Agent calls MCP tool (e.g., `show()`).
2. MCP server obtains a `DisplayClient` from the session’s connection registry, then calls `client.show(scene_id, elements)`.
3. Client serializes `SceneMessage` as JSON, frames with 4-byte big-endian length prefix, sends over Unix socket.
4. `SocketServer` (`src/punt_lux/socket\_server.py`) receives the frame in `poll_clients()`, deserializes via `FrameReader.drain_typed()`, dispatches to `DisplayServer.handle_message()`.
5. `DisplayServer` delegates state updates to `SceneManager.handle_scene()` or `handle_framed_scene()`.
6. `DisplayServer.render_scene()` draws the active scene each frame via `ElementRenderer`.
7. `DisplayServer` sends an `AckMessage` back to the client.

2.2.2 Event Path (User → Hub, two-tier D21)

The event path is the clearest place the migration shows through. The display no longer dispatches interactions locally or broadcasts a generic interaction message; it forwards the interaction back to the owning Hub, where the authoritative handler runs. This is the D21 two-tier model described in §3.

1. When a scene is installed, the display calls `elem.wrap_handlers_for_remote(_emit_event)` on each element, replacing every handler bucket with a `RemoteDispatchGroup` that captures the socket-send closure.
2. User interacts with a widget (button click, checkbox toggle). `ElementRenderer` fires the typed event on the display-side element copy; the wrapped handler runs instead of the real handler.
3. The wrapper builds one `RemoteEventHandlerInvocation` (`element_id`, `action`, `event_kind` — `button_clicked` or `value_changed` —, `value`, `ts`) and hands it to `_emit_event`, which stamps the current `scene_id` and queues it. One interaction produces one message per element/event bucket, however many handlers that bucket holds.
4. Each frame, `_flush_events()` records the events in the `QueryDispatcher` ring buffer (for `list_recent_events()`), then `_dispatch_queued_events()` routes each event to the *owning* client — the connection whose scene this is, resolved via `SceneManager.scene_to_owner` — falling back to broadcast only when no owner is known.
5. The owning Hub resolves the authoritative Element by (`scene_id`, `element_id`), validates ownership in `Display.interact()`, and fires the real event once. The original handler chain — `call_model`, `publish`, `mark_removed` — runs on the Hub's copy.

2.2.3 Update Path (Incremental Patching)

1. Agent calls `update(scene_id, patches)`.
2. `DisplayServer` receives `UpdateMessage`, delegates to `SceneManager.apply_update()`.
3. `_find_element()` (module helper in `scene/manager.py`) locates target by ID via depth-first search through the element tree.
4. Patch applied: `set` updates fields in-place, `remove` deletes element and cleans up widget state via `WidgetState.set(eid, None)` and `clear_suffix()`.
5. Routes to correct scene by `scene_id` (not limited to active tab); `SceneManager.resolve_scene()` looks up either the unframed-scenes dict or the per-frame scene dict.

3 The Hub/Display Slice (Migrated Core)

Everything above describes the legacy single-process path, still the mass of the shipped product. This section describes the migrated core — the part of the code that already implements the target model from `target/target.md`. It is narrow but load-bearing: it is where new element work lands, and it is the reference for how the whole system is meant to behave once migration completes.

The slice lives in `src/punt_lux/domain/` and imports no `ImGui`, no `socket`, and no `JSON` — it is pure domain. Adapters wrap it: the display process renders it, the Hub transport moves its serialized elements over the wire.

3.1 Two Authorities, One Winner

The target model splits the old display server into a **Hub** and a **Display** that are separate processes, potentially on separate machines.

- The **Hub** owns authoritative UI state and runs the real handlers. Clients (MCP tools, direct client APIs) talk to the Hub, never to the Display.
- The **Display** holds a full replica of the UI for rendering and input capture. It wraps handlers so interactions route back to the owning Hub instead of executing locally.

The rule is: **UI state crosses the wire; render calls do not.** Serialized elements, scene updates, and remote interaction messages travel between Hub and Display; ImGui calls stay local to the Display. When Hub-side state changes, the Hub re-sends the whole affected UI and the Display replaces its copy — there is no incremental diff protocol.

3.2 HubDisplay: the Authoritative Store

HubDisplay (`src/punt_lux/domain/hub/hub\_display.py`) is the Hub’s single authoritative surface for scene state. It is a facade over five typed collaborators, each with one responsibility:

Collaborator	Responsibility
ElementIndex	(<code>scene_id</code> , <code>element_id</code>) → <code>Element</code> lookup table.
OwnerTracker	Every <code>Element</code> ’s owning <code>ConnectionId</code> ; the interaction path gates on this, and the disconnect path walks it to find each connection’s owned roots.
RootRegistry	Scene-root ABC <code>Elements</code> that participate in the property-observer cascade.
ChildIndex	Parent → children edges captured at install time so cascade removal drops every descendant in one walk.
HubClientRegistry	The set of connections currently registered as Hub clients.

`apply(connection_id, update)` dispatches on the typed `Update` sum — `AddElement`, `SetProperty`, `RemoveElement` — and enforces ownership: a connection cannot mutate or evict another connection’s elements. `AddElement` recurses into composite children via the Composite Protocol so a `Button` buried inside a `Dialog` is indexed and later click-resolvable. `replace_scene` keeps the Hub authoritative on re-show: it removes the connection’s previous roots, then installs the new ones through the same `apply` path so ownership, root observers, and child indexes rebuild in one place.

3.3 The Element ABC

The migrated kinds subclass the `Element ABC` in `src/punt_lux/domain/element\_abc.py`. The ABC carries data *and* behavior:

- **Composite render template.** `render()` is a template method (never overridden); composites override `_children()` to return their children, leaves inherit the empty default.
- **Handler registry.** `add_handler` / `remove_handler` / `fire` key handlers by `Event` subclass. `fire` snapshots the bucket so a handler that mutates the registry mid-dispatch cannot affect the in-flight call.
- **Property-observer surface.** `add_observer`, `mark_removed`, and `removed` let a parent composite react to a child being removed. All three removal paths — agent `RemoveElement`, component self-dismiss, connection disconnect — route through the one `mark_removed` method.
- **Native serialization.** `__reduce__` / `__setstate__` let an `Element` cross the Hub→Display boundary. Hub-only bookkeeping (the observer closures that reference `HubDisplay`) is dropped on serialization; the Display is a replica and does not need Hub observers. Handlers survive so `wrap_handlers_for_remote` can wrap them on the Display side.

3.4 Two-Tier Handler Dispatch (D21)

The Hub says: “I am sending you a copy of a UI. If anyone interacts with it, I will do the work.” The Display says: “I will render that copy and forward interactions back to the Hub.”

Both tiers hold the same element state and both call `elem.fire(event)`. The difference is what the handler does:

- **Hub:** `elem.fire(event)` runs the real handler — `call_model`, `publish`, `mark_removed`.

- **Display:** `wrap_handlers_for_remote` has replaced each event bucket with a `RemoteDispatchGroup` (`src/punt_lux/domain/handlers/remote_dispatch.py`), so `elem.fire(event)` sends one `RemoteEventHandler` to the Hub. The Hub resolves the authoritative Element by `(scene_id, element_id)` in `Display.interact()`, validates ownership, and fires the real event once.

Two event types cross this boundary today: `ButtonClicked` and `ValueChanged` (`src/punt_lux/domain/interaction.py`). Grouping is per element/event bucket: a button with three handlers still produces a single remote message, and the Hub replays the full three-handler chain once on its copy. When a handler mutates Hub-side state (for example a dialog dismissing itself via `mark_removed`), the Hub re-pushes the full scene tree and the Display renders whatever the current tree is.

3.5 Agent Subscribe / Publish

The Hub also owns an application-level publish/subscribe channel (`src/punt_lux/domain/hub/hub.py`), distinct from the element-observer mechanism above — the two share no machinery.

Every operation is scoped to a `ConnectionId`. A connection cannot see, touch, or publish into another connection’s topics. `subscribe` registers the caller’s outbound writer against a topic; `publish` fans an `ObserverMessage` payload out to that connection’s subscribers using snapshot-then-iterate so a slow handler cannot stall concurrent publishes. Topics — `openTicket`, `closeTicket`, `markTicketInProgress` — are defined by the app or agent, not by Lux. This is the business-event channel; it is not the UI-state replication path.

The MCP surface exposes this channel through `subscribe`, `unsubscribe`, `publish`, and `recv` (§11).

3.6 Migrated vs. Still-Legacy

To keep the migration boundary honest:

State	What it covers
Migrated	The Element ABC and its four kinds (<code>text</code> , <code>button</code> , <code>checkbox</code> , <code>dialog</code>); <code>HubDisplay</code> authoritative state with ownership and cascade removal; the D21 <code>RemoteEventHandlerInvocation</code> two-tier event path; Agent Subscribe/Publish.
Transitional	The display process, which now runs the D21 event path (§6) yet still owns rendering, scene tabs, frames, and the query surface as in the legacy design.
Legacy	The 21 not-yet-migrated element kinds as frozen dataclasses; <code>SceneManager</code> multi-scene/frame state; <code>socket_server.py</code> single-process transport; the legacy ad-hoc introspection request/response pairs kept for compatibility.

The direction is fixed by `target/target.md`; the pace is set by migrating one element family and one subsystem at a time.

4 Wire Protocol

4.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----+
```

Maximum payload size: 16 MiB (2^{24} bytes). The `FrameReader` (in `src/punt_lux/protocol/\_\_init\_\_.py`) accumulates partial reads into a byte buffer and yields complete frames via `drain()` (raw dicts) or `drain_typed()` (Message dataclasses). Oversize frames raise `ValueError`, which `SocketServer` converts to a client disconnect.

4.2 Message Types

The message layer is split across six families: `scene`, `lifecycle`, `menu`, `introspect`, `observer`, and `remote_invocation`. There is no standalone interaction family — user interactions travel as `RemoteEventHandlerInvocation` from the `remote_invocation` module. In `src/punt_lux/protocol/messages/\_\_init\_\_.py`, the protocol surface is assembled into a `ClientMessage` union (12 request/command types), a `DisplayMessage` union (9 response/event types), and `UnknownMessage` for forward compatibility.

4.2.1 Client → Display

Type	Purpose
<code>SceneMessage</code>	Replace or add scene. Fields: <code>id</code> , <code>elements[]</code> , <code>title</code> , <code>layout</code> , <code>frame_id</code> , <code>frame_title</code> , <code>frame_size</code> , <code>frame_flags</code> , <code>frame_layout</code> .
<code>UpdateMessage</code>	Incremental patches. Fields: <code>scene_id</code> , <code>patches[]</code> (each: <code>id</code> , <code>set</code> , <code>remove</code>).
<code>ClearMessage</code>	Remove all scenes and reset state.
<code>PingMessage</code>	Heartbeat. Field: <code>ts</code> .
<code>ConnectMessage</code>	Client identity handshake. Field: <code>name</code> .
<code>MenuMessage</code>	Set agent-provided menu entries.
<code>ThemeMessage</code>	Apply theme by name.
<code>RegisterMenuMessage</code>	Register per-client menu items. Field: <code>items[]</code> .
<code>IntrospectRequest</code>	Request scene element tree (legacy ad-hoc op; retained alongside <code>QueryRequest</code> for backward compatibility). Field: <code>scene_id</code> .
<code>ListScenesRequest</code>	Request list of all active scenes (legacy ad-hoc op).
<code>ScreenshotRequest</code>	Request display screenshot (legacy ad-hoc op).
<code>QueryRequest</code>	Generic introspection/control request. Fields: <code>method</code> , <code>params</code> . The dispatcher routes by <code>method</code> string — see <code>introspection-api.md</code> .

SceneMessage frame fields (since v0.9.0): `SceneMessage` includes optional fields `frame_id`, `frame_title`, `frame_size` (initial `[width, height]`), `frame_flags` (dict of ImGui window flags such as `no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`), and `frame_layout` ("tab" or "stack").

4.2.2 Display → Client

Type	Purpose
<code>ReadyMessage</code>	Handshake on connect; protocol version, capabilities.
<code>AckMessage</code>	Acknowledges scene or update; <code>scene_id</code> , <code>ts</code> , optional <code>error</code> .
<code>RemoteEventHandlerInvocation</code>	User interaction forwarded to the owning Hub for authoritative dispatch; <code>element_id</code> , <code>action</code> , <code>event_kind</code> (<code>button_clicked</code> / <code>value_changed</code>), <code>value</code> , <code>ts</code> , optional <code>scene_id</code> .
<code>ObserverMessage</code>	Connection-scoped business event fan-out for the Agent Subscribe / Publish path; <code>topic</code> + <code>payload</code> .
<code>PongMessage</code>	Ping response; round-trip latency probe.
<code>IntrospectResponse</code>	Scene element tree response.
<code>ListScenesResponse</code>	List of active scene summaries plus frame summaries (<code>scenes[]</code> , <code>frames[]</code>).
<code>ScreenshotResponse</code>	Path to captured PNG.
<code>QueryResponse</code>	Generic introspection/control response. Fields: <code>method</code> , <code>result</code> , optional <code>error</code> .

Unknown message types are deserialized as `UnknownMessage` for forward compatibility rather than disconnecting the client.

The dispatch table (`MessageRegistry` in `src/punt_lux/protocol/messages/registry.py`) is populated at import time by `register_codecs` calls from each family module. Tests can construct an isolated registry without touching the module-level default — this replaces an earlier `if/elif` chain in `message_from_dict`.

4.3 Element Kinds

The wire layer exposes 25 element kinds plus the non-element `Patch` payload used by `UpdateMessage`. Four kinds — `text`, `button`, `checkbox`, `dialog` — have been migrated onto the `src/punt_lux/domain/element\_abc.py` `Element ABC`: they own data *and* behavior (a handler registry, the Composite render template, and the property-observer surface). The remaining kinds are still frozen, slotted dataclasses registered into `ElementCodec`. The `Element` union is assembled in `src/punt_lux/protocol/elements/\_\_init\_\_.py`.

On the encode side, `_element_to_dict` routes `text`, `button`, and `dialog` through the io-model `JsonEncoderFactory`; `checkbox` is ABC-backed but serializes through its own per-kind codec. On the decode side, `JsonElementFactory` reconstructs the io-model kinds with the tier-injected renderer factory and emit callback so handlers wire up on both Hub and Display.

Category	Element	Notes
basics	<code>text</code>	Content + style (heading, caption, success, error, code); optional <code>color</code> hex string
	<code>markdown</code>	CommonMark rendering via <code>imgui_md</code>
	<code>image</code>	File path or base64 data
	<code>separator</code>	Visual divider
	<code>progress</code>	Fraction bar with label
	<code>spinner</code>	Loading indicator
inputs	<code>button</code>	Label, action, disabled flag; <code>arrow</code> (directional) and <code>small</code> variants
	<code>slider</code>	Min/max/value, integer mode
	<code>checkbox</code>	Boolean toggle; the shipped interactive ABC element — a toggle fires <code>ValueChanged</code> , routed to the Hub as a <code>value_changed</code> invocation
	<code>combo</code>	Dropdown selection
	<code>input_text</code>	Single-line text input
	<code>radio</code>	Radio button group
	<code>input_number</code>	Numeric input with step buttons, min/max clamping, integer mode
	<code>color_picker</code>	RGB/RGBA hex picker; <code>alpha</code> (RGBA) and <code>picker</code> (full widget) modes
	<code>selectable</code>	Click-to-select item
dialog	<code>dialog</code>	Dialog root with title, children, handlers, and an internal model-backed lifecycle
layout	<code>group</code>	Row, column, or paged layout (<code>pages</code> + <code>page_source</code>)
	<code>tab_bar</code>	Tabbed container
	<code>collapsing_header</code>	Expandable section
	<code>window</code>	Movable, resizable floating panel
	<code>tree</code>	Recursive node hierarchy; <code>flat</code> flag for non-indented expansion
	<code>modal</code>	Modal popup dialog; container with children, emits "closed" event

Category	Element	Notes
graphics	draw	Canvas with typed draw commands (see §4.4)
	plot	ImPlot: line, scatter, bar series
table	table	Columns, rows, built-in filters, detail panel, pagination

4.4 Draw Commands

The `draw` element holds a tuple of typed `DrawCommand` records. Each command is a frozen, slotted dataclass that satisfies the `DrawCommand Protocol` (`TYPE: ClassVar[str]`, `to_wire(self) -> dict`, `from_wire(cls, d, *, ctx) -> Self`).

Module	Record	Geometry
draw_commands_line	Line	Start, end, color, thickness.
	Polyline	Point list, color, thickness, closed flag.
draw_commands_shape	Rect	Top-left, bottom-right, color, thickness, rounding, filled.
	Circle	Center, radius, color, thickness, filled.
	Triangle	Three points, color, thickness, filled.
draw_commands_curve	BezierCubic	Four control points, color, thickness.
draw_commands_text	TextGlyph	Position, content, color.

Shared value classes live in `draw_values.py` and `draw_bounds.py`: `Point2` (validated $[x, y]$ pair), `Color` (`#RRGGBB` / `#RRGGBBAA` hex), `Thickness` (strictly positive stroke width), `Radius` (strictly positive), `Rounding` (non-negative). The `WireContext` in `draw_wire.py` carries the per-error prefix (`draw command [i] (kind)`) and the require-or-raise helpers (`require_number`, `require_sequence`, etc.). `DrawCommandDecoder` in `draw_decoder.py` dispatches on the wire `cmd` string to the appropriate `from_wire` classmethod; `DrawElement.commands` is typed `tuple[DrawCommand, ...]` and the renderer dispatches via a typed `match` statement.

5 Display Server Internals

5.1 Frame Cycle

The display server runs an `ImGui` render loop via `hello_imgui.run()`. Each frame, the `DisplayServer.on_frame()` callback executes four phases in sequence:

1. **Accept connections.** `SocketServer.accept_connections()` runs `select()` on the server socket with timeout 0. Accept if readable. Send `ReadyMessage` to the new client.
2. **Poll clients.** `SocketServer.poll_clients()` runs `select()` on all client sockets with timeout 0. Read available data into the per-client `FrameReader`. Dispatch complete messages to `DisplayServer.handle_message()`, which routes by `isinstance` to the per-message-type handler. Remove errored sockets via `remove_client()`.
3. **Render scene.** Enable docking via `dock_space_over_viewport`. Render the idle flame background (`idle_screen.render_idle()`). Check for World menu background click. Render World panel if open. Render all workspace frames (see §6). Render the dock bar for minimized frames. Legacy fallback: if no frames exist, render unframed scenes as tabs.
4. **Flush events.** Route all buffered `RemoteEventHandlerInvocations` to each scene's owning client via `SocketServer.send_to_client()` (falling back to broadcast only when no owner is known), then clear the queue. Each event is also recorded in `QueryDispatcher.recent_events` for later `list.recent_events()` queries.

All four phases are non-blocking. The frame completes in microseconds when idle. `ImGui` handles frame pacing via `GLFW vsync`; `hello_imgui.fps.idling.fps_idle = 30.0` caps the idle frame rate.

5.2 Multi-Scene State

Scene state is owned by `SceneManager` in `src/punt_lux/scene/manager.py`, exposed to the rendering layer through read-only properties on `DisplayServer.scene_manager`.

Field	Type and Purpose
<code>_scenes</code>	<code>dict[str, SceneMessage]</code> — unframed scenes; ordered by insertion; keyed by <code>scene_id</code> .
<code>_scene_order</code>	<code>list[str]</code> — explicit tab ordering for unframed scenes.
<code>_active_tab</code>	<code>str None</code> — currently selected tab in the unframed-scenes view.
<code>_scene_widget_state</code>	<code>dict[str, WidgetState]</code> — per-scene slider/checkbox/combo/text state.
<code>_dirty_windows</code>	<code>set[str]</code> — window element IDs needing initial position set.
Frame state	
<code>_frames</code>	<code>dict[str, Frame]</code> — all workspace frames, keyed by <code>frame_id</code> .
<code>_focus_frame_id</code>	<code>str None</code> — frame to auto-focus on next render.
<code>_scene_to_frame</code>	<code>dict[str, str]</code> — scene → frame lookup.
<code>_scene_to_owner</code>	<code>dict[str, int]</code> — scene → client fd ownership.
Display-server state	
<code>_event_queue</code>	<code>list[RemoteEventHandlerInvocation]</code> on <code>DisplayServer</code> , drained each frame.
<code>_client_names</code>	On <code>SocketServer</code> , <code>fd</code> → display name from <code>ConnectMessage</code> .
<code>_menu_registrations</code>	On <code>MenuManager</code> , per-client agent menu items.

5.2.1 Scene Lifecycle

New scene (`scene_id` not in `_scenes`)

Appended to `_scenes` and `_scene_order`. Fresh `WidgetState` allocated. Becomes the active tab. Window elements marked dirty. Existing events preserved.

Replace-in-place (same `scene_id`)

Content replaced in `_scenes`. Widget state and render function state cleared for that scene. Events for elements that existed in the old scene but not the new scene are drained via `on_scene_replaced` callback (using full subtree ID collection through `_collect_ids()`). Events from other scenes are untouched.

Dismiss (tab close button)

Scene removed from all state dicts. Window element IDs removed from `_dirty_windows`. Active tab moves to the adjacent neighbor (next tab, or previous if rightmost was dismissed).

Clear (ClearMessage or menu)

All scenes, widget state, events, and dirty windows cleared. Fresh instances allocated (not `.clear()` on aliased objects).

5.2.2 Widget State Swap Pattern

The renderer's per-kind `ElementRenderer.RENDERERS` dispatch table all reads from `self._widget_state`. Rather than threading a `scene_id` parameter through every method, the display swaps the per-scene state dict into this instance variable before rendering each tab:

```
def _render_scene_tab(self, scene_id: str) -> None:
    self._element_renderer.widget_state = \
        self._scene_manager.scene_widget_state[scene_id]
    self._element_renderer.current_scene_id = scene_id
```

```

scene = self._scene_manager.scenes[scene_id]
for elem in scene.elements:
    self._element_renderer.render_element(elem)

```

This is the same pattern ImGui itself uses: “current context.” Zero changes required in any renderer method.

5.3 Table Filtering

Tables support built-in filtering at frame rate with zero MCP round-trips. `TableRenderer` (in `src/punt_lux/display/table/_renderer.py`) renders filter controls (search input, combo dropdowns) above the table and applies filters in the render loop:

1. Filter state stored in `WidgetState` with prefixed keys (e.g., `_tbl_search_{eid}`, `_tbl_filter_{col}_{eid}`).
2. `_apply_table_filters()` renders controls, snapshots state, detects changes.
3. `_filter_indexed_rows()` applies AND logic across all active filters (case-insensitive substring for search, exact match for combos).
4. Pagination resets to page 0 when filters change.
5. Detail panel renders a two-column metadata grid plus long-form body text for the selected row.

6 Frame Architecture

Since v0.9.0, scenes can target named *frames*—persistent ImGui windows that outlive client connections. Frames are the primary unit of workspace organization. The `Frame` class lives in `src/punt_lux/scene/frame.py`; `SceneManager` owns the `_frames` dict and the scene-to-frame routing.

6.1 Frame Class

Field	Type and Purpose
<code>frame_id</code>	<code>str</code> — Unique identifier.
<code>title</code>	<code>str</code> — Window title bar text.
<code>owner_fds</code>	<code>set[int]</code> — File descriptors of contributing clients.
<code>scenes</code>	<code>dict[str, SceneMessage]</code> — All scenes in this frame, keyed by <code>scene_id</code> .
<code>scene_order</code>	<code>list[str]</code> — Tab ordering.
<code>active_tab</code>	<code>str None</code> — Currently visible scene.
<code>minimized</code>	<code>bool</code> — Docked to bottom bar.
<code>cascade_index</code>	<code>int</code> — Stagger offset for initial positioning.
<code>initial_size</code>	<code>tuple[int,int] None</code> — Width/height hint (first use only).
<code>flags</code>	<code>dict[str,bool] None</code> — ImGui window flags (<code>no_resize</code> , <code>no_collapse</code> , <code>no_title_bar</code> , <code>no_background</code> , <code>no_scrollbar</code> , <code>auto_resize</code>).
<code>layout</code>	<code>Literal["tab", "stack"]</code> — Multi-scene composition mode.

Public read/write properties wrap each field; the constructor follows the PY-CC-1 `__new__` pattern.

6.2 Frame Lifecycle

Creation

A frame is created on the first `SceneMessage` with a `frame_id` not already in `_frames`. The client’s `fd` is added to `owner_fds`.

Shared ownership

Multiple clients can contribute scenes to the same frame. Each client that sends a scene targeting the frame is added to `owner_fds`.

Orphan model

When a client disconnects, its scenes persist with an `_ORPHAN_FD` sentinel (`-1`) in place of the real fd. The frame remains visible and interactive. A new client connecting with the same `frame_id` adopts the orphaned scenes.

Dismissal

The close button removes the frame and all its scenes from state. Widget state for all scenes in the frame is cleaned up.

Focus management

New frames are positioned using cascade staggering (`cascade_index` increments per frame). Initial size is applied on first render only. `_focus_frame_id` triggers `SetNextWindowFocus` on the next render cycle.

6.3 World Menu

Left-clicking the docking background opens the World menu panel, providing frame management operations: list all frames, restore minimized frames, close all frames, and access display settings.

6.4 Dock Bar

Minimized frames appear as clickable buttons in a bottom dock bar. Clicking a dock bar button restores the frame to its previous position and size.

7 IPC and Process Management

This section describes the legacy single-process transport: agents and apps connect to the display over a Unix socket, either directly or through higher-level client wrappers, and `luxd` acts as a WebSocket/session gateway. In this path scene storage, widget state, event routing, and rendering all belong to the display process. The target topology relocates authoritative scene state into the Hub (§3); the socket mechanics below are what the migration is moving away from.

7.1 Socket Discovery

`DisplayPaths` in `src/punt_lux/paths.py` resolves the display socket path in priority order:

1. `$LUX_SOCKET` environment variable.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.
3. `/tmp/lux-$USER/display.sock` (fallback).

The fallback path is chosen to stay within macOS's ~104-character `AF_UNIX` path limit.

The hub socket and PID files are resolved by module-level helpers in the same file: `hub_dir()`, `hub_pid_path()`, `hub_port_path()`, `hub_log_dir()`, `read_hub_port()`, `is_hub_running()`.

7.2 PID Lifecycle

- PID file written at `{socket_path}.pid` on server startup.
- Clients verify liveness via `os.kill(pid, 0)` before connecting.
- Stale socket cleanup: `DisplayPaths.cleanup_stale()` removes socket and PID file if the process is dead.

7.3 Auto-Spawn

`DisplayPaths.ensure()` spawns the display server if not running:

1. Checks socket existence and PID liveness.
2. If not running, spawns
`python -m punt_lux display --socket {path}`
with `start_new_session=True` (detached from caller's process group).
3. Polls socket existence + PID liveness every 100 ms until ready or 5 s timeout.
4. Display logs to `{socket_path}.log`.

7.4 Tool Reconnect Path

The MCP surface wraps display-bound calls in `client_registry.with_reconnect()`: if an `OSError` (for example a broken pipe) occurs, the stale client is closed, a new connection is established, and the call is retried exactly once. There is no unbounded retry loop.

8 Performance

8.1 Frame Budget

Parameter	Value	Notes
Idle FPS	30	<code>fps_idling.fps_idle = 30.0</code>
Active FPS	60+	GLFW vsync, GPU-bound
Socket poll timeout	0	Non-blocking <code>select()</code>
Max message size	16 MiB	Hard limit in <code>FrameReader</code>
Connection timeout	5.0 s	<code>ensure()</code> and handshake
Texture loading	Lazy	First render triggers PIL + OpenGL upload
Event ring buffer	200 entries	<code>QueryDispatcher._recent_events</code>
Error ring buffer	100 entries	<code>QueryDispatcher._recent_errors</code>

8.2 Critical Path Analysis

The frame-critical path is:

1. `select()` on N client sockets — $O(N)$, $N < 64$ in practice.
2. `FrameReader.feed()` — memcpy into buffer, yield frames. $O(B)$ where B is bytes received.
3. `DisplayServer._render_scene()` — ImGui draw list calls via `ElementRenderer.render_element()`. $O(E)$ where E is total element count across visible scene.
4. `_flush_events()` — send K events to N clients. $O(K \cdot N)$.

Table filtering is $O(R \cdot C)$ where R is row count and C is column count. This runs every frame but is fast for tables under $\sim 10,000$ rows. No benchmarks exist for larger tables; pagination (configurable page size) bounds visible row rendering.

8.3 Idle Screen

The idle screen renders ~ 60 draw calls per frame: 48 radial lines, 3 glow circles, 3 flame shapes (each ~ 7 bezier operations). This is negligible for a GPU-rendered immediate-mode UI at 30 FPS idle.

8.4 Memory

- **Scene storage.** Scenes are Python dataclass trees held in memory. No size limit beyond the 16 MiB message cap. Each scene is a separate dict entry; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs held in `TextureCache` until `_on_exit()`. No eviction policy. Image-heavy scenes accumulate GPU memory.
- **Widget state.** Per-scene `WidgetState` is a flat dict. Cleared on scene replacement. No growth concern.
- **Event queue.** Flushed every frame. Bounded by interaction rate (<100 events/frame in practice).
- **Ring buffers.** `deque(maxlen=200)` for events and `deque(maxlen=100)` for errors; constant memory.

8.5 Known Limitations

1. **No ImGui ID scoping.** Widget IDs are derived from `element_id` alone, not scoped by scene. ImGui internal state (focus, open/closed) can bleed between tabs with colliding IDs.
2. **No texture eviction.** Long-running sessions with many images accumulate GPU memory.
3. **Filter is per-frame.** Table filtering re-executes every frame even when nothing changed. Not a problem at current scale but would benefit from dirty-flag optimization for 10,000+ row tables.

9 Security

9.1 Threat Model

The primary trust boundary is the Unix socket. The display server accepts connections from any process that can reach the socket file. In the default configuration (`/tmp/lux-$USER/`), the socket is protected by filesystem permissions (user-only directory, mode 0700).

Threat	Description	Mitigation
Socket hijacking	Another process connects to the socket and injects scenes or reads events.	Directory permissions (0700) on <code>/tmp/lux-\$USER/</code> .
Malformed messages	Client sends invalid JSON, unknown message type, or oversize payload.	Unknown types deserialized as <code>UnknownMessage</code> ; oversize payloads rejected by <code>FrameReader</code> .
Render function	Agent-submitted Python code executes arbitrary operations.	Consent dialog with AST warnings. User must explicitly click Allow.
Resource exhaustion	Scene with millions of elements or very large images.	16 MiB message cap. No element count limit (ImGui handles gracefully).

9.2 Render Function Security

Render functions are the only element kind that executes arbitrary code. The security model is **consent-based, not sandbox-based**:

1. **AST scanning:** a best-effort visitor flags suspicious imports (`os`, `subprocess`, `socket`, `ctypes`, `urllib`, etc.) and dangerous builtins (`eval`, `exec`, `open`, `__import__`). This is a *UX signal*, not a security boundary.
2. **Consent dialog:** an ImGui modal displays the full source code with line numbers and any AST warnings. The user must click “Allow” to proceed. Dismissing the modal (Esc) is treated as “Deny.” Consent is per-element: changing the source code resets consent.

3. **Execution:** `CodeExecutor` compiles the source and calls `render(ctx)` each frame. The `RenderContext` provides a persistent state dict, delta time, frame counter, canvas dimensions, and a `send()` callback for emitting events. Execution is **not sandboxed**: the code runs in the display server’s process with full access to the Python runtime. Runtime errors are caught and displayed, not propagated.

Implication: render functions should be treated as user-approved code, equivalent to running a script the user downloaded. The consent dialog provides informed consent, not isolation.

9.3 Protocol Robustness

- **Malformed JSON:** client disconnected, not crashed. `SocketServer` logs the error via the `on_error` callback and removes the client.
- **Unknown message type:** deserialized as `UnknownMessage` for forward compatibility.
- **Oversize frame:** `FrameReader` checks `payload_len` against `MAX_MESSAGE_SIZE` and raises; `SocketServer` converts to disconnect.
- **Partial reads:** `FrameReader` accumulates bytes across frames; no assumption of complete messages per `recv()`.
- **Identity fields protected:** `SceneManager.apply_update()` silently drops patches that attempt to change `id` or `kind` on an element.
- **Double-remove idempotent:** `SocketServer.remove_client()` is safe to call twice on the same socket.

10 Platform Integration

10.1 macOS

- **Dock presence.** The display server runs as a normal macOS Dock app per GLFW’s default activation policy (Regular). The Dock icon and Cmd-Tab entry are intentionally visible to aid operational debugging.
- **Process name.** `setproctitle("Lux")` makes the process identifiable in `ps` and `top`. Does not affect Activity Monitor’s binary name column.
- **Font discovery.** Prefers Arial Unicode or Helvetica; merges Apple Symbols for mathematical/Z notation glyphs and STIX Two Math for Mathematical Alphanumeric Symbols (U+1D400–1D7FF).
- **AF_UNIX path limit.** macOS limits Unix socket paths to ~104 characters. `tempfile.mkdtemp(prefix="lux-")` used in tests; production paths are short by design.

10.2 Linux

- **Font discovery.** DejaVu Sans or Noto Sans; merges Noto Sans Symbols2 and Noto Sans Math for Unicode coverage.
- **XDG compliance.** Prefers `$XDG_RUNTIME_DIR/lux/` for socket placement.
- **No Dock equivalent.** Window managers use the X11 window title (set to “Lux”).

11 MCP Tool Surface

The display-facing MCP surface is registered primarily in `src/punt_lux/tools/tools.py`. Simple scene and configuration tools are declared directly with `@mcp.tool()`; query-style operations route through the generic `QueryRequest` envelope where possible. Separate session-pub-sub tools live in `src/punt_lux/tools/subscribe\_tools.py` and are distinct from the display’s UI interaction stream.

Tool	Signature and Purpose
Scene management	
<code>show</code>	<code>(scene_id, elements, title?, layout?, frame_id?, frame.title?, frame.size?, frame.flags?, frame.layout?)</code> — Display a scene built from element dicts.
<code>show_table</code>	<code>(scene_id, columns, rows, filters?, detail?, flags?, title?, frame_id?, frame.title?)</code> — Convenience wrapper for a filterable table scene.
<code>show_dashboard</code>	<code>(scene_id, metrics?, charts?, table.columns?, table.rows?, title?, frame_id?, frame.title?)</code> — Convenience wrapper for metrics, charts, and a summary table.
<code>update</code>	<code>(scene_id, patches)</code> — Incremental JSON patches targeting elements by ID.
<code>clear</code>	<code>()</code> — Clear all scenes.
Display control	
<code>ping</code>	<code>()</code> — Latency probe; returns <code>PongMessage</code> .
<code>set_menu</code>	<code>(menus)</code> — Set agent-provided menu bar entries.
<code>register_tool</code>	<code>(label, tool_id, shortcut?, icon?)</code> — Register an item in the shared Lux Tools menu.
<code>set_theme</code>	<code>(theme)</code> — Apply display theme. Routes through <code>QueryRequest</code> .
Display configuration	
<code>set_window_settings</code>	<code>(opacity?, font_scale?, decorated?, fps_idle?)</code> — Configure display window appearance and idle frame rate.
<code>set_frame_state</code>	<code>(frame_id, minimized?)</code> — Control frame state (minimize/restore).
Display introspection	
<code>inspect_scene</code>	<code>(scene_id)</code> — Element tree for a scene. The MCP helper routes through <code>QueryRequest</code> ; the legacy <code>IntrospectRequest</code> message remains in the wire protocol for compatibility.
<code>list_scenes</code>	<code>()</code> — All active scenes plus frame summaries. The MCP helper also routes through <code>QueryRequest</code> .
<code>screenshot</code>	<code>()</code> — Intended to capture the display as a PNG. The wire protocol retains a dedicated <code>ScreenshotRequest/ScreenshotResponse</code> path; the current MCP helper is transitional and still reaches the display through the query surface.
<code>get_display_info</code>	<code>()</code> — Display dimensions, frame count, client count, FPS, uptime, PID.
<code>get_window_settings</code>	<code>()</code> — Opacity, font scale, decorated, idle FPS.
<code>get_theme</code>	<code>()</code> — Current theme and available themes.
<code>list_clients</code>	<code>()</code> — Connected clients with display names and connect times.

Tool	Signature and Purpose
<code>list_menus</code>	<code>()</code> — Registered menu items, per client.
<code>list_recent_events</code>	<code>(count?)</code> — Recent interaction events. Reads <code>QueryDispatcher._recent_events</code> (max 200).
<code>list_errors</code>	<code>(count?)</code> — Recent error log entries. Reads <code>QueryDispatcher._recent_errors</code> (max 100).
Per-repo config helpers	
<code>display_mode</code>	<code>(repo)</code> — Read the repo-local display mode from <code><repo>/punt-labs/lux.md</code> .
<code>set_display_mode</code>	<code>(mode, repo)</code> — Write the repo-local display mode and eagerly connect when enabling.
Session pub-sub helpers	
<code>subscribe</code>	<code>(topic)</code> — Subscribe the calling session to a topic within its own scope.
<code>unsubscribe</code>	<code>(topic)</code> — Remove the calling session’s subscription to a topic.
<code>publish</code>	<code>(topic, payload?)</code> — Fan out a business event to subscribers in the calling session’s scope.
<code>recv</code>	<code>(timeout?)</code> — Wait for the next session-scoped business event. This is <i>not</i> the display UI interaction stream.

The introspection surface is split between the generic `QueryRequest/QueryResponse` envelope and a small number of older dedicated request/response pairs kept for compatibility. At the protocol level, `screenshot` still expects the dedicated `ScreenshotRequest/ScreenshotResponse` path.

Six handlers are built into `QueryDispatcher` at construction time: `inspect_scene`, `list_scenes`, `list_clients`, `list_menus`, `list_recent_events`, `list_errors`.

Seven more handlers are registered by `DisplayServer` at construction time because they touch ImGui state: `screenshot` (stub — raises), `get_display_info`, `get_window_settings`, `get_theme`, `set_window_settings`, `set_frame_state`, `set_theme`.

12 Formal Model

The legacy display server’s state machine is formally specified in `docs/architecture/display-server.tex` using the Z notation (ISO 13568). The specification models the single-process path, and its refinement tests still hold the current display code to that model. It covers:

- Client connection/disconnection with `FrameReader` association.
- Scene replacement, incremental patching, and clearing.
- Interaction event generation and broadcast.
- `FrameReader` streaming buffer invariants.

The specification has been type-checked with Spivey’s `fuzz` and model-checked with ProB (7,943 reachable states explored). Refinement tests in `tests/test_display_refinement.py` verify commutativity between the abstract Z operations and the concrete Python implementation via an abstraction function (`tests/display_abstraction.py`).

Partition tests in `tests/test_display_partition.py` derive test cases from the Z specification using the Test Template Framework (TTF), covering all input space partitions of each operation schema.

Note: the formal model predates multi-scene tabs and models a single-scene display. The abstraction function maps multi-scene state to the active tab for backward compatibility. A specification update to model the full multi-scene state is pending.

13 Test Architecture

The test suite is broad and layered; the exact count changes regularly, so this document records the coverage shape rather than a brittle inventory snapshot.

- **Protocol and codec tests:** framing, serialization, typed draw-command decode, value validation, and query envelopes. Representative files: `test_protocol.py`, `test_draw_*`, `test_query.py`.
- **Display-state and manager tests:** pure state transitions for the display server without a live GUI. Representative files: `test_display_state.py`, `test_scene_manager.py`, `test_query_dispatcher.py`, `test_socket_server.py`.
- **Tooling and client tests:** `test_tools.py`, `test_display_client.py`, `test_show.py`, `test_cli.py`, `test_paths.py`, `test_service.py`, `test_remote.py`, and `test_config.py` cover the MCP surface, client wrappers, CLI, process helpers, remote config, and repo-local configuration.
- **Formal-model alignment:** `test_display_refinement.py` and `test_display_partition.py` tie the concrete Python implementation back to the Z specification.
- **Marker-gated integration and e2e tests:** `test_integration.py`, the integration-marked cases in `test_query.py`, and `test_e2e.py` exercise real sockets and, for the e2e path, a live display environment.

Quality gates are driven by the Makefile: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest` (default marker set), `check-oo` (OO ratchet against `.oo-baseline.json`), and `check-suppressions`. `make check` is the composite gate.